

Lab 1 - Profiling Parallel Code (2nd part)

Advanced Computer Architectures

University of Minho

The Lab 1 focus on the development of efficient CPU code by revisiting the programming principles that have a relevant impact on performance, such as vectorisation and scalability of multithreaded algorithms. Submit jobs to the mei queue in SeARCH to measure code execution times, but be careful to always use the same node architecture (i.e., the nodes in the *cpar* partition).

This lab tutorial includes three exercises to be solved during the lab class (Lab 2.x) and a suggested additional exercise (Ext 2.4).

Consider a set of simple microbenchmarks to measure different characteristics of a system, available in the SeARCH cluster. Note that these are not industry standard benchmarks; they are simplified versions that do not achieve the same performance, but their code is easier to understand. You can copy it to your home directory by:

```
cp -r /share/cpar/ACA/lab1-2 .
```

To load the compiler in the environment use one of the following commands (GCC is advised):

Intel VTune: `module load vtune/2020`

GNU Compiler: `module load gcc/11.2.0`

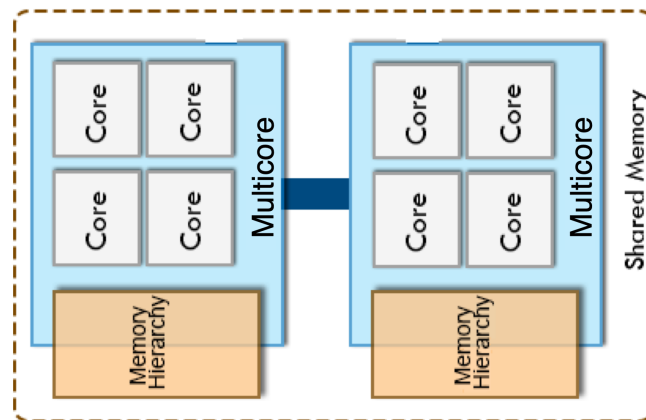
Remember that this must also be done inside a script if you are not using the compute node interactively. All performance measurements for the entire session must be documented.

2 Characterising a computing system

Goals: to develop skills in the fundamentals of system benchmarking.

Lab 2.1 Consider the architecture of the Ivy Bridge computing nodes available in the SeARCH cluster.

You will need to assess the performance of this system in terms of its computing capabilities. This enables the understanding of the limits of the system, and how far a given code is from these limits. They can be assessed theoretically - extremely complex and error prone - or by experimentation through specialised benchmarks. They must be individually compiled using their respective Makefiles.



Start by measuring the maximum computing throughput of (i) a single core and a (ii) single CPU. Choose the adequate microbenchmark from the ones available in the `lab1-2` folder. Compile and submit the benchmark to be executed on the cluster using the `job.sh` script (experiment with different amounts of threads!):

```
sbatch job.sh
```

The outputs should appear on two files, once the job is executed: `slurm-job-id.out` and `slurm-job-id.err`. You can check the status of your job through the command `squeue -u username`. Record the results that are output by the benchmark.

Do you get the same performance assessment of the system if you run the same benchmark with a VTune analysis?

Lab 2.2 A high computing throughput often requires a large system bandwidth to feed the algorithm with the necessary data. Choose the adequate microbenchmark to measure the bandwidth of a single CPU accessing its memory bank, compile and submit it to be executed. Experiment with different amounts of threads and dataset sizes.

Is this the bandwidth for the whole system? If not, can you calculate it theoretically based on the measurements performed?

Lab 2.3 The microbenchmarks from the previous exercises operate on data structures that are allocated on the memory bank of the CPU that executes the code. However, there is a performance penalty of accessing data that is stored in the other CPU (see Non-Unified Memory Accesses). Since this behaviour may occur on real-world code, especially when parallel code uses both CPUs but carelessly allocates data structures, its performance degradation must be understood.

Rerun the microbenchmarks of the previous exercises to assess the impact on computing and memory throughput of allocating data in the improper memory bank. Record the results.

Lab/Ext 2.4 Consider the Social Media Analyser code from the previous lab session. Choose a workload size that ensures that the application executes for at least 5 seconds and measure its computing and memory throughput with VTune - choose the relevant analyses based on your experience from the previous session.

How far is the measured computing and memory bandwidth from the maximum experimental limits from the previous exercises? Is the code memory- or compute-bound?